

Desenvolvimento de uma aplicação *web* para gerenciamento de trabalhos acadêmicos

Paulo Ricardo Savaris dos Santos

Orientador: Kleyton Hoffmann

Universidade do Oeste de Santa Catarina - Unoesc

CET - Área de Ciências Exatas e Tecnológicas

Curso de Engenharia de Computação - Campus de Joaçaba/SC

Resumo—¹ Este artigo propõe o estudo e desenvolvimento de uma aplicação *web* baseada na metodologia ágil Kanban para o gerenciamento de Trabalhos de Conclusão de Curso (TCC), visando ser utilizada como ferramenta de apoio e monitoramento durante o processo de elaboração do TCC, para otimizar seu desenvolvimento e gerenciamento. Realizou-se uma pesquisa com estudantes e professores do ensino superior para avaliar a viabilidade do trabalho, identificando as dificuldades na organização dos TCCs e a necessidade de uma abordagem mais eficiente no gerenciamento. Com base nos resultados obtidos, tornou-se evidente uma aprovação de mais de 95% em relação à utilização de uma aplicação *web* para o gerenciamento dos TCCs. Essa expressiva aceitação foi um impulso para estudar metodologias ágeis, com foco especial no Kanban, que serviu como referencial teórico para o desenvolvimento da aplicação proposta. Foram criados diagramas de atividades para compreender o funcionamento do sistema e protótipos de tela, a fim de garantir a compatibilidade e funcionalidade a aplicação com a metodologia ágil, fornecendo orientação durante o desenvolvimento da aplicação. A aplicação objetiva contribuir para a melhoria da qualidade e eficiência na elaboração dos TCCs, proporcionando um ambiente de trabalho mais ágil e produtivo. Através da simplificação da interação entre alunos e orientadores, a aplicação busca criar um ambiente colaborativo que otimize o fluxo de trabalho, facilite o acompanhamento das etapas do projeto e assegure o cumprimento dos prazos estabelecidos.

Palavras chave—Palavras-chave: Aplicação *web*, gerenciamento de projetos, Kanban, metodologia ágil, Trabalho de Conclusão de Curso, TCC.

Abstract—This article proposes the study and development of a web application based on the agile Kanban methodology for the management of Final Course Projects (FCP), aiming to be used as a support and monitoring tool during the process of FCP elaboration, to optimize its development and management. A survey was conducted with high school students and professors to evaluate the viability of the work, identifying the difficulties in organizing FCPs and the need for a more efficient approach in management. Based on the obtained results, an approval exceeding 95% regarding the use of a web application for managing FCPs became evident. This significant acceptance propelled the study of agile methodologies, with special focus on Kanban, which served as a theoretical reference for the development of the proposed application. Activity diagrams were created to understand the functioning of the system and screen prototypes, in order to guarantee the compatibility and functionality of the application with the agile methodology, providing guidance during the development of the application. Objective application contributes

to improving the quality and efficiency in the elaboration of FCPs, providing a more agile and productive work environment. By simplifying the interaction between students and advisors, the application seeks to create a collaborative environment that optimizes the workflow, facilitates the monitoring of the stages of the project and ensures compliance with the established deadlines.

Index Terms—Keywords: Agile methodology, Final Course Project, FCP, Kanban, project management web application.

I. INTRODUÇÃO

O trabalho de conclusão de curso, conhecido como TCC, é um trabalho acadêmico exigido em diversas instituições de ensino como requisito para obtenção do certificado de curso superior, elaborado em forma de dissertação e pode ser realizado de várias formas, sendo comumente exigido no formato monografia ou artigo.

O desenvolvimento do TCC ocorre sob a orientação de um professor integrante do corpo docente da universidade e perpassa algumas etapas, quais sejam:

- 1) Elaboração de um pré-projeto;
- 2) Estruturação do projeto;
- 3) Execução da pesquisa;
- 4) Redação de conclusão.

No pré-projeto, o estudante, com auxílio do orientador, delimita o tema a ser pesquisado. Na sequência, ocorre a estruturação do projeto, na qual são identificadas a problemática, a justificativa, a metodologia a ser utilizada e as hipóteses de conclusão. Na terceira etapa é realizada a pesquisa e, por fim, é feita a redação do trabalho, com o relatório detalhado e explicativo sobre o desenvolvimento e as conclusões da pesquisa realizada.

Contudo, apesar de ter as etapas bem delimitadas, o desenvolvimento do trabalho de conclusão de curso pode ser um desafio para muitos estudantes, principalmente em razão da dificuldade de organização e desenvolvimento do trabalho, além da administração do tempo. É importante ressaltar que a maioria dos graduandos conciliam trabalho, vida pessoal e vida estudantil, o que pode ocasionar, por vezes, a má gestão do tempo e, por via de consequência, a reprovação do acadêmico.

Posto isso, o presente trabalho tem como objetivo o estudo e desenvolvimento de uma aplicação *web* para ser utilizada como ferramenta de apoio e monitoramento no processo de elaboração do trabalho de conclusão de curso, a fim de otimizar o desenvolvimento e gerenciamento da atividade final.

¹Trabalho de Conclusão de Curso apresentado ao Curso de Engenharia de Computação no segundo semestre de 2023, Área de Ciências Exatas e Tecnológicas da Universidade do Oeste de Santa Catarina como requisito parcial à obtenção do grau de bacharel em Engenharia de Computação.

Foi elaborado uma aplicação intuitiva e de fácil utilização, que norteie o estudante, mediante a disponibilização de uma estrutura básica, conforme o formato de trabalho exigido, que também auxilia a não esquecer elementos imprescindíveis, bem como permitira verificar o progresso no desenvolvimento, com a identificação das etapas concluídas, tarefas pendentes e prazos a serem observados.

Além disso, a aplicação permite que o orientador acompanhe o desenvolvimento do projeto simultaneamente ao acadêmico, possibilitando a criação de tarefas e sub-tarefas, realizem movimentações e alterações e deixem bilhetes com dúvidas ou orientações. Dessa forma, será possível manter o controle das atividades, identificar problemas e resolvê-los de maneira proativa, visando a otimização do trabalho e uma gestão mais eficaz de tempo e recursos, além de facilitar a comunicação entre orientador e orientado.

A utilização dessa aplicação *web* traz benefícios significativos para a comunidade acadêmica como um todo. Ao facilitar o gerenciamento de trabalhos, contribui para um melhor aproveitamento do tempo e recursos disponíveis, resultando em trabalhos com mais qualidade e maior produtividade por parte dos estudantes e orientadores.

Neste trabalho, são abordados conceitos e tecnologias relacionadas ao desenvolvimento *web*, bem como os princípios e boas práticas para a criação de uma aplicação eficiente e de fácil utilização. É apresentada uma revisão dos trabalhos relacionados, com enfoque em aplicações semelhantes, como o Trello e o Wrike, analisando como essas ferramentas abordam o gerenciamento de projetos e trabalhos acadêmicos.

A. Aplicações Relacionadas

Existe uma grande quantidade de ferramentas disponíveis no mercado para o gerenciamento de projetos. No presente trabalho serão apresentadas duas aplicações comumente utilizadas: Trello e Wrike.

O primeiro destaca-se pela simplicidade e facilidade de utilização. Sua *interface* limpa e intuitiva permite que os usuários comecem a fazer uso da ferramenta rapidamente, sem a necessidade de treinamento. Além disso, o Trello oferece uma visão clara das tarefas a serem desenvolvidas, possibilitando que a equipe visualize facilmente o que precisa ser feito, as prioridades e a responsabilidade de cada membro. A aplicação dispõe ainda de recursos de colaboração em tempo real, permitindo que os membros da equipe trabalhem juntos, compartilhem ideias e atualizem o status das tarefas de forma fácil e rápida [1].

Por outro lado, o Wrike oferece vantagens para equipes em busca de uma solução de gerenciamento de projetos robusta e flexível. Sua *interface* intuitiva e de fácil utilização permite que os usuários se adaptem rapidamente à plataforma e colaborem de forma eficaz. Com recursos avançados de colaboração, como compartilhamento de arquivos, discussões em tempo real e atribuição de tarefas, o Wrike facilita o trabalho eficiente e coordenado da equipe. A flexibilidade no gerenciamento de tarefas é outro destaque, permitindo que os usuários personalizem suas tarefas de acordo com as preferências individuais e acompanhem o progresso em tempo real. Além disso, a aplicação oferece integrações com outras

ferramentas populares, simplificando a sincronização de dados e o fluxo de trabalho entre diferentes plataformas.

É importante mencionar que o Wrike oferece um período de teste gratuito de 14 dias, após o qual o usuário precisa assinar um dos planos disponíveis para continuar utilizando a aplicação.

A presente aplicação visa facilitar ao máximo a interação entre os usuários e seus respectivos Trabalhos de Conclusão de Curso (TCCs) diferenciando-se das outras aplicações mencionadas anteriormente por sua prioridade em oferecer uma interface intuitiva, simples e objetiva.

II. FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica engloba estudos sobre o processo de elaboração de um Trabalho de Conclusão de Curso e as metodologias mais utilizadas, a fim de proporcionar uma base sólida para o desenvolvimento da aplicação proposta.

Este trabalho propõe o desenvolvimento de uma aplicação *web* voltada para o gerenciamento de trabalhos de conclusão de curso (TCC), utilizando como base a metodologia ágil Kanban. Para embasar essa proposta, são abordados conceitos relacionados à metodologia ágil, o Manifesto Ágil e seus princípios, bem como os fundamentos do Kanban como uma abordagem visual para o gerenciamento de projetos. Além disso, são explorados os conceitos e tecnologias envolvidas no desenvolvimento da aplicação, tais como Spring Boot, como projeto de desenvolvimento, PostgreSQL como sistema de gerenciamento de banco de dados, e o Figma como ferramenta de *design* para o levantamento de requisitos e prototipação da interface da aplicação.

A. TCC e sua metodologia

O Trabalho de Conclusão de Curso (TCC) é uma importante produção acadêmica que os estudantes precisam desenvolver para concluir o curso, seja de graduação ou pós-graduação. Ele representa a síntese do conhecimento adquirido ao longo da formação acadêmica e serve como uma avaliação dos conjuntos de conhecimento do estudante. Os trabalhos acadêmicos geralmente seguem normas científicas, como a ABNT, Vancouver, APA, entre outros métodos de execução [2, 3].

Cada instituição de ensino possui suas próprias normas para a elaboração do TCC, que deve contribuir cientificamente para a sociedade. Desta forma é necessário que o formando interaja com outras áreas, realize pesquisas bibliográficas e de campo, a fim de elaborar um trabalho que possa solucionar um problema.

O TCC é obrigatório e geralmente é desenvolvido e apresentado nos dois últimos semestres do curso. O acompanhamento de um professor-orientador é fundamental para o TCC, pois o professor irá orientar o estudante na busca de referências teóricas e aprofundamento do tema [3, 2].

Existem várias formas de se receber uma certificação, como monografias, relatórios de estágio, artigos científicos, dissertações de mestrado e teses de doutorado. O escopo e formato do TCC podem variar entre diferentes cursos e instituições [4].

Após a definição do tema, o estudante deve elaborar uma pergunta ou problema de pesquisa e justificar a escolha do tema. Os objetivos gerais e específicos devem ser estabelecidos,

assim como a metodologia de pesquisa, incluindo local, coleta e análise de dados [2].

De acordo com Araujo [5] “ metodologia científica representa muito mais que formatação de trabalhos acadêmicos e sim, todo o seu desenvolvimento, desde a definição da pesquisa, até a redação final.” O planejamento adequado da pesquisa, seguindo fases e garantindo a gestão temporal e de recursos, auxilia na conquista dos objetivos finais.

A metodologia científica e o gerenciamento de projetos estão interligados. Enquanto a primeira define o método de pesquisa para obter resultados confiáveis e comprováveis, o gerenciamento de projetos envolve processos, métodos e ferramentas para alcançar os objetivos do projeto.

No caso do trabalho em questão, o objetivo é desenvolver uma aplicação para apoio e monitoramento de elaboração do TCC, otimizando o desenvolvimento, o gerenciamento e a comunicação entre orientador e aluno.

B. Gerenciamento de projetos.

Segundo o PMI [6] um projeto “é um esforço temporário empreendido para criar um produto, serviço ou resultado exclusivo”. Sendo assim, um projeto pode ser entendido como um esforço para alcançar um objetivo bem definido em um determinado período de tempo.

O gerenciamento de projetos é uma disciplina muito importante para projetos em qualquer área, incluindo trabalhos acadêmicos como o Trabalho de Conclusão de Curso (TCC). Segundo Arnaldo Fernandes e Muyllder [7], “o sucesso de um projeto depende de um gerenciamento adequado, que envolve planejamento, controle e execução eficientes”.

O uso de ferramentas e técnicas para gerenciamento de projetos pode trazer diversos benefícios para diversos indivíduos, em especial acadêmicos em fase de elaboração do trabalho de curso, pois permitem uma gestão mais efetiva do tempo, recursos e riscos envolvidos no projeto, além de promover uma comunicação mais clara e eficaz entre o orientando e o orientador.

No contexto do TCC, a utilização de ferramentas de gerenciamento de projetos pode ajudar os estudantes a se manterem organizados, a definir e cumprir prazos, a evitar retrabalho e a garantir a qualidade do trabalho entregue. Além disso, a adotar uma abordagem sistemática para o gerenciar o trabalho pode contribuir para uma análise mais completa e rigorosa dos objetos de estudo, resultando em pesquisas mais consistentes e confiáveis.

Nesse sentido, é importante lembrar que existem diversas metodologias de gerenciamento de projetos disponíveis atualmente, cada uma com suas particularidades e vantagens. Entre elas, destacam-se as metodologias ágeis, como o Kanban, que vêm se tornando cada vez mais utilizado em diversas áreas.

C. Manifesto ágil

O Manifesto Ágil foi criado em 2001 por um grupo de 17 profissionais de *software* durante uma reunião, com o objetivo de encontrar uma abordagem mais flexível, colaborativa e adaptativa para o desenvolvimento de *software*. Esses profissionais representavam metodologias já utilizadas, como SCRUM, Extreme Programming, DSDM, entre outras [8].

De acordo com Carvalho Sbrocco e Paulo Cesar de Macedo [9] “O Manifesto Ágil foi criado considerando a velocidade demandada para novos sistemas de *software*, com o objetivo de abandonar métodos antigos que se mostravam ultrapassados. Trata-se de um documento que encoraja o uso de melhores métodos de desenvolver sistemas de *software*.” Esse documento estabelece os valores e princípios que norteiam e encoraja o uso de métodos para o desenvolvimento ágil de *software*.

Além de ser complementado por 12 princípios que oferecem orientação prática para a implementação do desenvolvimento ágil o Manifesto Ágil enfatiza quatro valores principais descritos na íntegra a seguir [8]:

- Indivíduos e interações mais que processos e ferramentas.
- *Software* em funcionamento mais que documentação abrangente.
- Colaboração com o cliente mais que negociação de contratos.
- Responder a mudanças mais que seguir um plano.

Sendo assim, o Manifesto Ágil representa uma base sólida para as metodologias ágeis que não se limitam apenas ao desenvolvimento de *software*, mas também podem ser utilizadas em outras áreas, como o gerenciamento de projetos, processos e rotinas.

A abordagem ágil destaca a importância de indivíduos e interações, trabalho em equipe, entrega contínua de valor e adaptação às mudanças. É uma abordagem flexível e colaborativa que valoriza a satisfação do cliente e a entrega de resultados tangíveis, fornecendo orientação prática para a adoção de metodologias ágeis em qualquer área, promovendo a eficiência, a qualidade e a inovação em todos os projetos [10].

c.1) Metodologias ágeis

Conforme Rehkopf [11] “As metodologias ágeis são um conjunto de ideais e princípios voltados para a melhoria do processo de desenvolvimento de *software*.” Elas são caracterizadas por serem estruturadas e iterativas à gestão de projetos e ao desenvolvimento de produtos, reconhecem a volatilidade do desenvolvimento de produtos e proporciona uma metodologia para equipes auto-organizadas responderem à mudança sem perder o rumo.

Já a definição de metodologia proposta por Alistair Cockburn [12] sugere que uma metodologia é o conjunto de convenções que uma equipe concorda em seguir. Ou seja, cada equipe pode desenvolver sua própria metodologia, que será diferente ou ligeiramente semelhante da metodologia de outras equipes.

Assim, tem-se que as metodologias ágeis são práticas seguidas pelas equipes para aderir aos valores e princípios do Manifesto Ágil, com o objetivo principal de promover a flexibilidade, a colaboração e a adaptabilidade, permitindo que as equipes atinjam seus objetivos de forma eficaz.

Vale ressaltar que a aplicação das metodologias ágeis pode variar de acordo com a equipe e o contexto, de modo que cada uma pode adaptar e personalizar as práticas ágeis de acordo com suas necessidades e características.

As metodologias ágeis estão se tornando cada vez mais relevantes, por exemplo na área de gerenciamento de projetos. Muitas equipes estão adotando princípios ágeis para se tornarem mais flexíveis e adaptáveis às mudanças constantes que ocorrem durante o ciclo de vida de desenvolvimento de um projeto. Isso tem levado a uma abordagem mais iterativa e incremental, permitindo a entrega de valor de forma mais rápida e eficiente [12].

c.2) Kanban

Dentre as metodologias ágeis destaca-se o Quadro Kanban desenvolvido pela Toyota na década de 1940, visando aumentar a produtividade da empresa a partir de um sistema prático, visual e simplificado. Posteriormente essa metodologia foi adotada em outras áreas de produção.

Em 2010, David J. Anderson adaptou o modelo para a área de desenvolvimento de *software*, sendo utilizado inicialmente na manufatura e, posteriormente, em outros contextos, incluindo o gerenciamento de projetos. O programa possui uma abordagem enxuta, dinâmica e leve para o gerenciamento de fluxo de atividades e se destaca pela abordagem visual e colaborativa de administração do fluxo de trabalho [1].

A metodologia ágil Kanban é uma abordagem de gestão visual que tem sido amplamente adotada em diversos contextos, como desenvolvimento de *software* e gerenciamento de projetos. O Kanban enfatiza a limitação do *Work in Progress* (WIP) ou na tradução literal Trabalho em progresso e a criação de um fluxo contínuo de entrega de valor [13].

De acordo com Anderson [13], “o Kanban é fundamentado em um conceito simples: o trabalho deve ser limitado e algo novo só deve ser iniciado quando uma peça de trabalho existente é concluída ou puxada por uma função subsequente.”

Seu funcionamento é baseado no uso de cartões (kanbans) para representar o trabalho em diferentes etapas do processo. Cada etapa possui um limite de trabalho em progresso, e o avanço é controlado pelo movimento dos cartões entre as etapas. Isso dá às equipes uma visão geral clara de seus fluxos de trabalho e as permite identificar gargalos e atrasos de forma rápida e eficiente [13].

Assim, a metodologia Kanban é uma abordagem flexível e colaborativa no gerenciamento de projetos e apresenta benefícios como maior transparência, flexibilidade e capacidade de resposta às mudanças.

A visualização do fluxo de trabalho facilita a identificação de gargalos e oportunidades de melhoria, permitindo que a equipe tome ações corretivas de forma proativa. Além disso, o Kanban promove a colaboração e o compartilhamento de conhecimento, incentivando a responsabilidade coletiva pela entrega de valor [13, 1, 14].

D. Spring

O Spring é um *framework* de código aberto que revolucionou o desenvolvimento Java ao simplificar a criação de aplicativos robustos. Oferece uma ampla gama de funcionalidades que abrangem desde a configuração até o acesso a dados e a integração de sistemas [15, 16].

Uma das características fundamentais do Spring é a sua capacidade de gerenciar dependências, possibilitando a inversão de controle e a injeção de dependências. Esses conceitos permitem que os desenvolvedores desacoplem componentes dos aplicativos, tornando-os mais testáveis, flexíveis e fáceis de manter [15, 16].

Além disso, o Spring fornece uma camada de abstração consistente sobre tecnologias existentes, simplificando tarefas comuns, como acesso a banco de dados, integração de serviços *web* e gerenciamento de transações. Sua modularidade também permite que os desenvolvedores escolham os módulos específicos do Spring que melhor se adequam às necessidades de seus projetos [15, 16].

E. Spring boot

Spring Boot é um projeto *open-source* de desenvolvimento de aplicativos Java, que visa simplificar e agilizar o processo de criação de aplicativos. Ele fornece um conjunto de ferramentas e convenções que permitem aos desenvolvedores iniciar rapidamente projetos, facilitando a configuração e o desenvolvimento [17, 18].

Uma das principais características do Spring Boot é sua abordagem de “convenção sobre configuração”, que significa que ele fornece configurações padrão úteis, permitindo que os desenvolvedores se concentrem na lógica de negócios, ao invés de se preocuparem com a configuração detalhada do aplicativo, o que, por conseguinte, resulta em uma maior produtividade e rapidez no desenvolvimento [17].

Além disso, o Spring Boot possui um sistema de gerenciamento de dependências simplificado, permitindo que os desenvolvedores adicionem facilmente bibliotecas e *frameworks* ao projeto por meio de ferramentas como Maven ou Gradle. Isso reduz a complexidade de configuração manual e facilita a incorporação de recursos adicionais [19, 17].

F. Thymeleaf

Para o *front-end* foi utilizado o Thymeleaf sendo esse um mecanismo de *template* para aplicações Java que permite a integração eficiente de dados dinâmicos no HTML. Utilizado principalmente em projetos Spring Boot, simplifica a criação de páginas *web* dinâmicas, permitindo a incorporação de expressões Thymeleaf diretamente no código HTML. Essas expressões facilitam a manipulação de dados do lado do servidor, tornando a apresentação de informações mais dinâmica e personalizada. [20]

G. JavaScript

Ao desenvolver páginas *web* interativas, a utilização de JavaScript em conjunto com HTML é uma prática comum. O JavaScript é uma linguagem de programação amplamente suportada pelos navegadores e permite a manipulação dinâmica da interface do usuário. Incorporado diretamente no HTML, possibilita a execução de *scripts* que respondem a eventos do usuário, realizam validações e atualizam o conteúdo da página de forma assíncrona. A combinação de JavaScript e HTML é essencial para criar experiências de usuário ricas e interativas. [21]

H. PostgreSQL

Desenvolvido com base no POSTGRES 4.2, na Universidade da Califórnia, no Departamento de Ciências da Computação em Berkeley, o PostgreSQL é um sistema gerenciador de banco de dados objeto-relacional de código aberto e altamente confiável. Compatível com os principais sistemas operacionais, o PostgreSQL segue os princípios ACID (Atomicidade, Consistência, Isolamento e Durabilidade) para transações, afim de garantir a confiabilidade e a integridade dos dados [22].

PostgreSQL tem como uma das principais características a capacidade de processar grandes volumes de dados com eficiência sem sacrificar a velocidade ou a estabilidade do sistema. Além disso é altamente personalizável e possui uma variedade de extensões e *plug-ins* disponíveis para atender às necessidades específicas do usuário. Por exemplo: o desenvolvedor pode definir seus próprios tipos de dados, criar funções personalizadas e até mesmo escrever código de diferentes linguagens de programação sem recompilar o banco de dados. O PostgreSQL também oferece suporte nativo para JSON e XML, tornando-o uma escolha popular para aplicativos *web* [22, 23].

O PostgreSQL tem forte ênfase na segurança, oferecendo uma ampla gama de recursos de segurança, como autenticação, SSL e criptografia de dados, motivo pelo qual é muito utilizado em aplicativos financeiros e governamentais.

No mais, a sua comunidade ativa e engajada, composta por desenvolvedores, especialistas e usuários, é conhecida por fornecer suporte e desenvolvimento contínuo, pois são empenhadas em melhorar o sistema, corrigir problemas e fornecer orientação aos usuários. Além disso, o PostgreSQL disponibiliza uma documentação abrangente e de alta qualidade, que facilita o aprendizado e o uso do sistema [23].

I. Figma

O Figma é uma ferramenta de *design* que combina a acessibilidade da *web* com as funcionalidades de um aplicativo nativo. Ele se apresenta como um editor *online* que traz todas as funcionalidades de um aplicativo tradicional [24].

Criado com o objetivo de tornar a prática do *design* mais acessível, oferece uma versão gratuita para permitir que os usuários conheçam a ferramenta e entrem na área de *design* sem a necessidade de grandes investimentos em programas similares [24].

Além disso, o Figma oferece recursos como prototipagem interativa, *design* responsivo e um sistema de componentes reutilizáveis, que agilizam o processo de *design* e garantem consistência visual em diferentes telas e dispositivos[25].

Por ser uma aplicação *web* e *online*, os projetos criados e editados no Figma são salvos automaticamente na nuvem. Isso permite que os usuários compartilhem facilmente seus projetos por meio de *links*, possibilitando o trabalho colaborativo em equipe. Além disso, é possível inspecionar os arquivos de *design*, permitindo que trechos de código gerados para CSS, iOS e Android sejam extraídos, o que facilita o trabalho dos desenvolvedores, pois eles não precisam criar arquivos CSS do zero para implementar o *design* criado [25].

III. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos é uma das atividades mais importantes no desenvolvimento de um *software*, pois permite saber sobre as necessidades do usuário e do ambiente em que o sistema será utilizado. Essa fase inicial do processo de desenvolvimento é fundamental para garantir o sucesso do projeto pois é a partir dela que se define o escopo do projeto e se estabelece o que será entregue.

Segundo [26] o levantamento de requisitos é um processo que envolve a coleta de informações sobre o sistema a ser desenvolvido e a definição dos requisitos que o sistema deve satisfazer, podendo ser feita por meio de entrevistas, questionários, análise de documentos e outras técnicas. É importante ressaltar que o levantamento de requisitos deve ser uma atividade iterativa/contínua, pois as necessidades do usuário e do ambiente podem mudar ao longo do tempo.

Com o objetivo de levantar os requisitos do sistema, foram desenvolvidos dois questionários. O primeiro questionário teve como propósito mapear as dificuldades enfrentadas pelos estudantes ao realizar um trabalho acadêmico, em especial o Trabalho de Conclusão de Curso. Além disso, buscou-se identificar quais ferramentas eles conheciam ou utilizavam para gerenciar seus trabalhos e qual formato de visualização de tarefas mais lhes agradava.

Já o segundo questionário teve como foco compreender o ponto de vista dos orientadores. Buscou-se entender quais aspectos eles consideram importantes em uma aplicação desse tipo, quais ferramentas utilizam, quais são as dificuldades enfrentadas e como visualizam o progresso dos trabalhos.

O questionário voltado para os orientandos contou com a participação de 24 estudantes, dos quais aproximadamente 58% relataram ter dificuldades em organizar ou fazer trabalhos acadêmicos, como mostrado na Figura 1.

Tem dificuldade em fazer/organizar trabalhos acadêmicos?

24 respostas

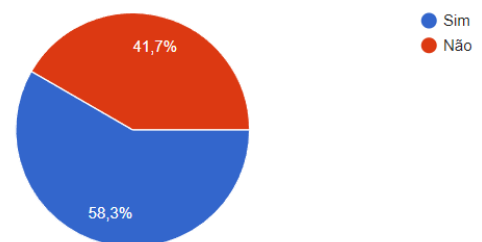


Fig. 1. Questionário - dificuldade em fazer/organizar trabalhos

Também foi observado que a grande maioria dos estudantes enfrenta dificuldades em organizar os trabalhos e gerenciar o tempo, sendo que 75% dos entrevistados relataram dificuldade em cumprir prazos, conforme evidenciado na Figura 2. Além disso, mais de 50% dos participantes conhecem ou utilizam a aplicação *web* Trello, sendo que mais de 70% preferem uma visualização de tarefas no formato de quadros, conforme visto na Figura 3, conhecido como método Kanban, utilizado pelo Trello.

Tem dificuldade em cumprir prazos?

24 respostas

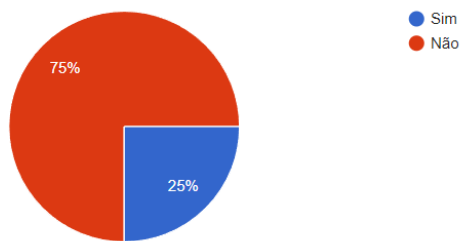


Fig. 2. Questionário - dificuldade em cumprir prazos

Acharia útil uma ferramenta para auxiliar em trabalhos acadêmicos? como por exemplo o TCC

24 respostas

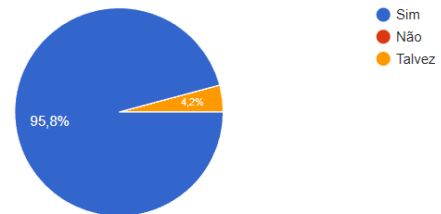


Fig. 5. Questionário - Viabilidade do trabalho

Qual o tipo de organização na gestão do trabalho acha melhor de visualizar as tarefas?

24 respostas

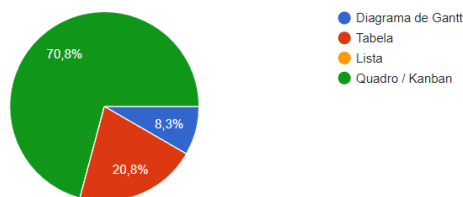


Fig. 3. Questionário - Preferencia de visualização da organização de tarefas

No questionário direcionado aos orientadores, nota-se que a maioria deles já utilizou alguma ferramenta para gerenciar trabalhos ou projetos. Foi possível perceber a necessidade de uma maior comunicação e interação com os orientandos, para que estes compreendam a complexidade, dificuldade, necessidade de tempo e atenção que um trabalho do porte de um TCC requer. Observou-se o uso de ferramentas colaborativas, como o Trello e o Overleaf, um editor de LaTeX *online*, enfatizando-se o uso desse tipo de ferramenta para um acompanhamento mais eficaz e um tempo de resposta mais ágil.

Questionou-se também qual tipo de aplicação mais agrada ao público-alvo e se uma aplicação voltada para o gerenciamento do TCC seria útil ou bem recebida. Mais de 58% dos entrevistados preferem uma aplicação *web*, conforme figura 4, e mais de 95% consideram viável uma ferramenta que auxilie nos trabalhos acadêmicos, conforme Figura 5, tornando assim a proposta deste trabalho viável.

Qual o tipo da aplicação mais agrada?

24 respostas

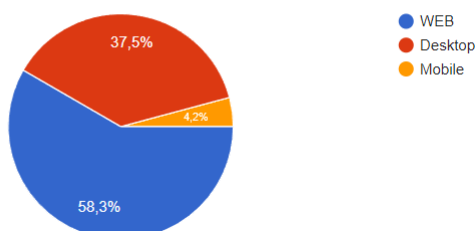


Fig. 4. Questionário - Tipo de aplicação mais votada

IV. METODOLOGIA

A pesquisa adota uma abordagem científica aplicada para a criação de uma aplicação *web* exploratória. O método utilizado é o hipotético-dedutivo, baseado no conhecimento de uma problemática existente no gerenciamento da elaboração do Trabalho de Conclusão de Curso e na dificuldade de comunicação entre aluno e orientador, com o objetivo de desenvolver uma ferramenta que otimize e agilize o desenvolvimento de trabalhos acadêmicos.

Os procedimentos adotados iniciam-se com uma pesquisa bibliográfica para embasar a formulação do projeto de trabalho, utilizando fontes doutrinárias e publicações relevantes. Além disso, foram utilizados dados documentais. O foco deste trabalho está na concepção e desenvolvimento de uma aplicação *web* que auxilie no planejamento da rotina e das tarefas do trabalho de conclusão de curso, utilizando como base a metodologia ágil Kanban para o gerenciamento das tarefas.

A. Visão Geral do Projeto

Inicialmente, a aplicação *web* e o banco de dados são alocados em um servidor local conhecido como *localhost*. Essa escolha facilita o desenvolvimento, monitoramento e comunicação por meio do protocolo HTTP. O *localhost* refere-se à máquina local que permite iniciar um servidor *web* no sistema, possibilitando testar novos programas sem a necessidade de um servidor externo.

Para acessar a aplicação, o usuário precisa fazer o login. Caso não possua uma conta, é necessário realizar um cadastro. Após o *login*, o usuário é direcionado para a tela de *Dashboard*, que oferece informações gerais sobre os trabalhos e tarefas. A partir da *Dashboard*, é possível acessar as telas de Perfil, *Dashboard* dos trabalhos, tela do trabalho e *Dashboard* das tarefas. Essas telas podem ser acessadas de forma intercambiável, conforme indicado pelas conexões na Figura 6.

Para acessar as telas de trabalho, o usuário terá duas opções: acessar a *Dashboard* de trabalhos primeiro e, em seguida, selecionar o trabalho desejado, ou utilizar um submenu que lista os trabalhos disponíveis. A criação de um novo trabalho só será possível se o usuário passar pela tela de *Dashboard* dos Trabalhos. O mesmo procedimento se aplica às tarefas, onde o usuário precisa acessar a tela do trabalho para criar e mover uma tarefa. Para convidar um novo membro, o processo é semelhante ao das tarefas, exigindo o acesso à tela do trabalho antes de ter acesso à tela de convite de membros.

Na Figura 6, apresenta-se um diagrama que ilustra a estrutura do projeto proposto. Esse diagrama permite visualizar a interconexão entre os componentes, os meios de comunicação e uma visão geral das transições entre telas, criação de trabalhos e tarefas no sistema.

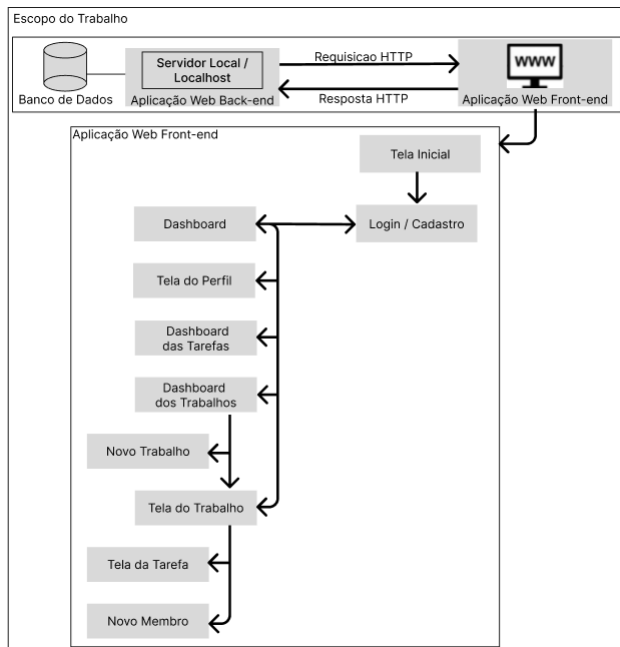


Fig. 6. Diagrama Geral do Projeto

Para o armazenamento eficiente de dados, foi utilizado o banco de dados PostgreSQL, mencionado anteriormente na fundamentação teórica. O PostgreSQL oferece vantagens como confiabilidade, escalabilidade e compatibilidade com padrões SQL, tornando-o uma escolha sólida para a aplicação.

A figura 7 apresenta o diagrama de entidade e relacionamento no qual representa a estrutura das tabelas no banco de dados

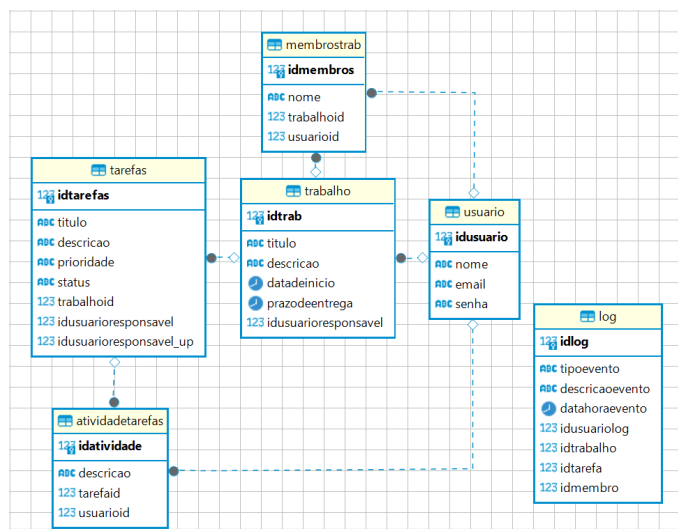


Fig. 7. Diagrama de relacionamento de entidades

Para o desenvolvimento das telas, foi utilizado o projeto Spring Boot em conjunto com as linguagens Java. O Spring

Boot é conhecido por sua robustez e facilidade de configuração, além de fornecer recursos que agilizam o desenvolvimento de aplicativos *web*.

Combinando o Spring Boot com as linguagens mencionadas, será possível criar telas interativas e responsivas. Utilizando HTML e CSS para estruturar e estilizar as páginas, proporcionando uma experiência de usuário agradável e intuitiva.

Essa combinação de tecnologias permitirá o desenvolvimento de uma aplicação *web* dinâmica, que atenda de forma eficiente e intuitiva às necessidades de gerenciamento de trabalhos de conclusão de curso.

B. Diagramas de Atividades

Os diagramas de atividades desempenham um papel importante na modelagem de sistemas utilizando a Linguagem de Modelagem Unificada (UML). Pois descrevem as ações e sequências necessárias para o funcionamento adequado do sistema.[27]

Na UML, um diagrama de atividade possibilita visualizar o fluxo de ações em uma atividade, sendo semelhante a um fluxograma. Essa representação gráfica auxilia na detecção de possíveis problemas ou gargalos no processo e auxilia no planejamento e implementação do sistema além de facilitar a compreender os passos e as interações entre as atividades, permitindo uma análise mais precisa do comportamento do sistema. Durante a implementação, esses diagramas fornecem uma orientação sobre as atividades e suas sequências, permitindo uma implementação eficiente e consistente [28].

Alguns elementos comuns em um diagrama de atividade incluem ações, nós de decisão, fluxos de controle, nó inicial e nó final.

A seguir são apresentadas algumas das principais funcionalidades do sistema através do diagrama de atividades. Sendo elas:

- Login/cadastro.
- Criação de trabalho.
- Adicionar membros.
- Criação de tarefas.

b.1) Cadastro e Login

A Figura 8 apresenta o diagrama de atividades para o cadastro de um novo usuário e de *login* no sistema.

O processo de cadastro é descrito da seguinte forma:

- O usuário preenche os dados necessários para o cadastro.
- Após preencher os dados, o sistema verifica se existem dados semelhantes já cadastrados.
- Se não houver dados semelhantes, o sistema prossegue com o cadastro e armazena as informações fornecidas.
- No caso de existirem dados semelhantes cadastrados, o sistema nega o cadastro.
- O usuário deve preencher o cadastro com outras informações para poder prosseguir.

O processo de *login* é descrito da seguinte forma:

- O usuário preenche os dados necessários para o *login*.
- Após o preenchimento dos dados, o sistema verifica se existem dados semelhantes já cadastrados para efetuar o *login*.

- Se houver dados semelhantes, o sistema permite o *login*, concluindo o ciclo e redirecionando o usuário para outra página.
- No caso de não existirem dados semelhantes cadastrados, o sistema nega o *login*.

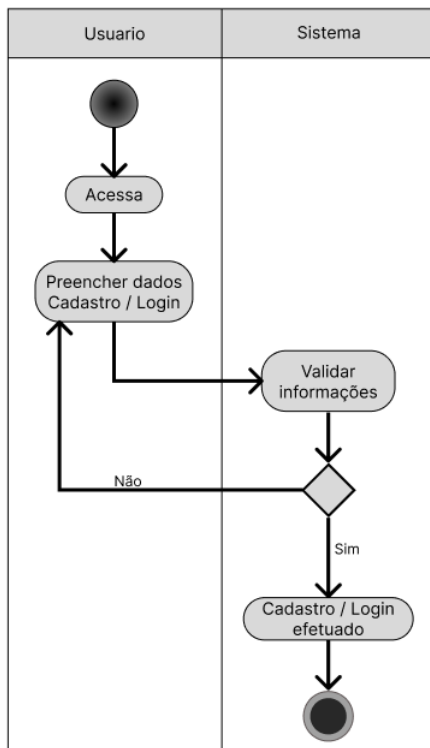


Fig. 8. Diagrama de Atividades - Cadastro e *Login*.

b.2) Criação de Trabalho

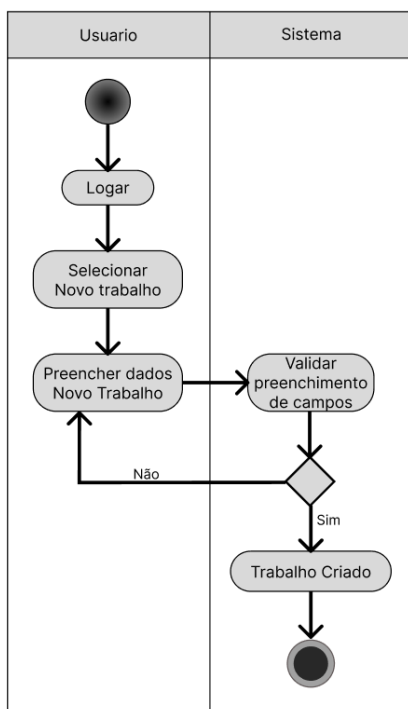


Fig. 9. Diagrama de Atividades - Trabalho

A Figura 9 representa o diagrama de atividades de criação de trabalho no sistema. O processo de criação de trabalho é descrito da seguinte forma:

- O usuário realiza *login*.
- Na tela de Trabalhos, seleciona a opção "Novo Trabalho".
- Caso algum campo obrigatório não seja preenchido, o sistema impede o salvamento e solicita novamente o preenchimento dos dados.
- Se os dados forem preenchidos corretamente, o sistema salva o trabalho criado e finaliza o ciclo.

b.3) Adicionar Membro

A Figura 10 representa o diagrama de atividades de adição de um novo membro em um trabalho no sistema. O processo de adição de membro é descrito da seguinte forma:

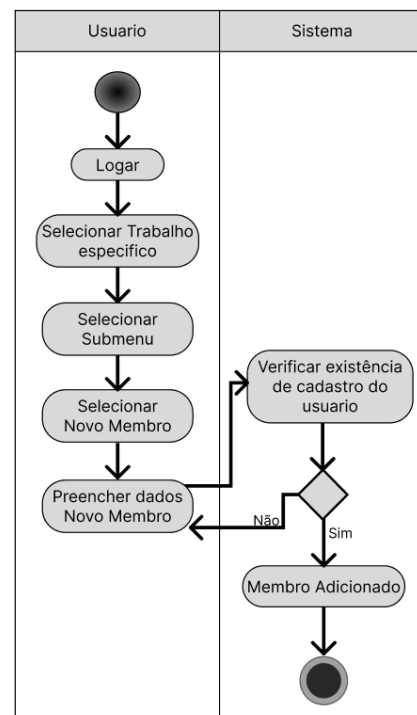


Fig. 10. Diagrama de Atividades - Membro

- O usuário realiza *login*.
- Na tela de Trabalhos, escolhe o trabalho em que deseja convidar o novo membro.
- Na tela do trabalho selecionado, escolhe a opção "Novo Membro".
- Se o usuário a ser convidado não estiver cadastrado ou houver preenchimento incorreto, o sistema impede o salvamento e solicita novamente o preenchimento dos dados.
- Se os dados forem preenchidos corretamente, o sistema salva a adição do membro e finaliza o ciclo.

b.4) Criação de Tarefas

A Figura 22 representa o diagrama de atividades de criação de tarefas em um trabalho no sistema. O processo de criação de tarefas é descrito da seguinte forma:

- O usuário realiza *login*.

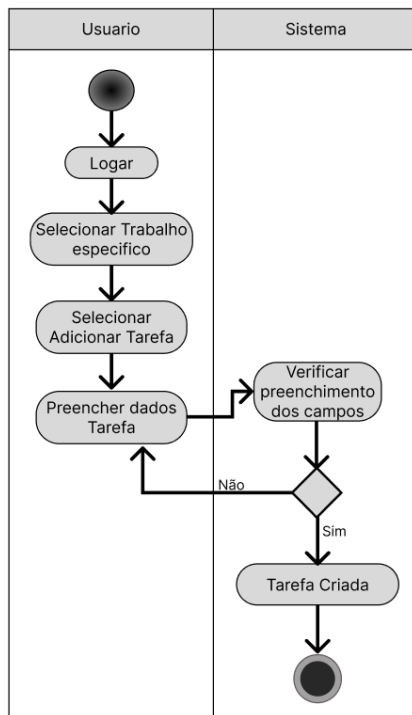


Fig. 11. Diagrama de Atividades - Tarefas

- Na tela de Trabalhos, escolhe o trabalho em que deseja criar uma nova tarefa.
- Na tela do trabalho selecionado, escolhe a opção "Adicionar Tarefa".
- Caso algum campo obrigatório não seja preenchido, o sistema impede o salvamento e solicita novamente o preenchimento dos dados.
- Se os dados forem preenchidos corretamente, o sistema salva a tarefa criada e finaliza o ciclo.

C. Protótipos de Telas do Sistema

A criação dos protótipos de telas do sistema iniciou-se com a utilização da metodologia Kanban e da plataforma *online* Trello como base. Além disso, optou-se pelo uso do editor Figma para o desenvolvimento dos protótipos.

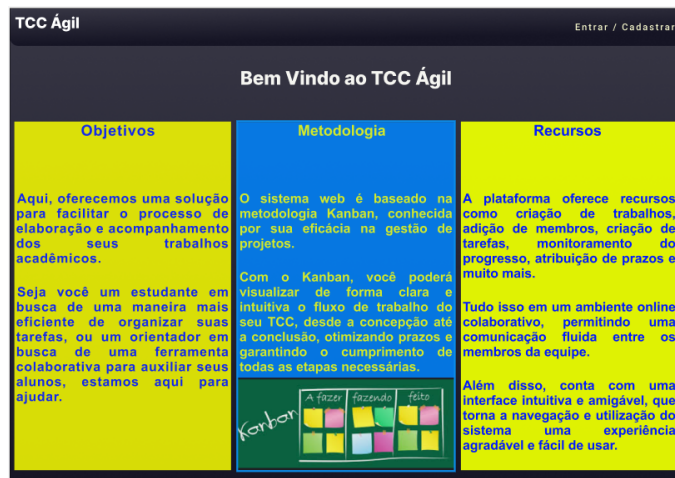


Fig. 12. Tela de Boas Vindas / Inicial

A tela inicial do sistema é a tela de boas-vindas (Figura 12), que fornece informações sobre os objetivos, metodologias e ferramentas da aplicação.

A tela representada na Figura 13 trata-se da tela de *login* e de cadastro. Nessa tela, é possível realizar o cadastro caso o usuário não possua uma conta. É importante destacar que durante o processo de cadastro, o usuário deve selecionar o tipo de perfil, identificando-se como orientador ou orientando.

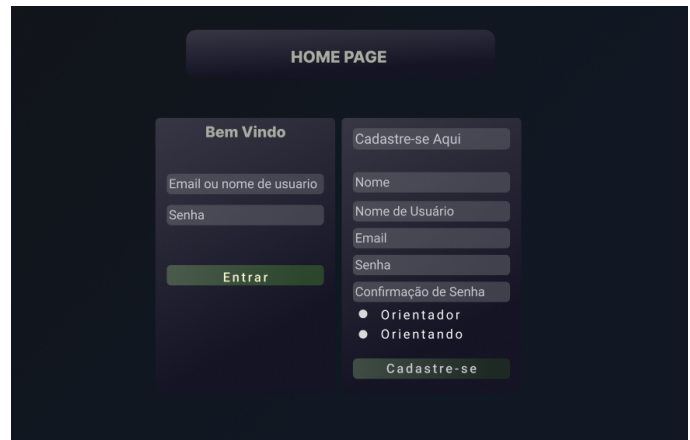


Fig. 13. Tela de Login e de Cadastro

Após efetuar o login, o usuário é redirecionado para a tela de *dashboard*, que varia de acordo com o perfil do usuário. Na Figura 14, é apresentado um exemplo de tela *Dashboard* para o perfil de orientador. É importante ressaltar que existe uma diferença entre as telas, sendo que no perfil de orientador é exibido um campo de Orientandos, enquanto no perfil de orientando esse campo não está disponível.

A tela de *dashboard* tem como objetivo principal ser um painel visual que apresenta um conjunto de indicadores relevantes para o usuário. Isso inclui informações como a quantidade de tarefas e trabalhos nos quais o usuário está envolvido. Esses indicadores fornecem uma visão geral das atividades e ajudam o usuário a gerenciar sua participação no sistema de forma mais eficiente.

Para facilitar a navegação entre as telas do sistema, foi implementado um menu à esquerda, o qual redireciona o usuário para diferentes indicadores, como *Dashboard*, Trabalhos e Tarefas, conforme descrito no menu. No campo de Trabalhos, há um ícone em forma de seta que, ao ser pressionado, permite ao usuário visualizar os trabalhos nos quais está participando.



Fig. 14. Tela de Dashboard.

Ao selecionar o campo Trabalhos no menu à esquerda, o usuário é redirecionado para a tela de Trabalhos, onde é apresentado um *dashboard* adicional contendo informações sobre

os trabalhos. Por exemplo, é possível visualizar a quantidade de membros participantes, o número de tarefas e um gráfico que indica, em porcentagem, os diferentes *status* das tarefas. A Figura 15 ilustra um exemplo da tela em que o Trabalho 1 possui 4 tarefas distribuídas entre os *status* “A fazer”, “Fazendo”, “Aguardando Retorno” e “Finalizada”, conforme evidenciado no gráfico logo abaixo onde é exibido o número de tarefas.

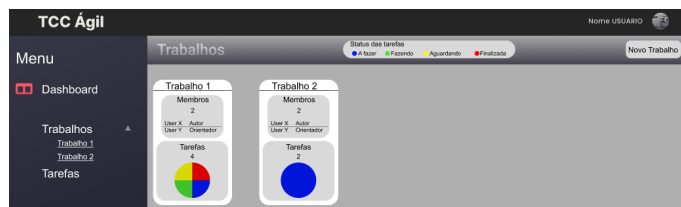


Fig. 15. Tela de *Dashboard* dos Trabalhos

Na mesma tela, à direita, encontra-se um campo denominado “Novo Trabalho”, o qual possibilita a criação de um novo trabalho. Ao pressionar esse campo, o usuário é redirecionado para uma tela de criação de trabalho, representada na Figura 16, na qual são solicitados os campos obrigatórios: Título e Descrição. Após o preenchimento desses campos, o usuário deve pressionar o botão “Criar”, localizado logo abaixo, para efetuar a criação do trabalho. Após a conclusão desse processo, o usuário é redirecionado para a tela do Trabalho.

Fig. 16. Tela de Novo Trabalho.

Na tela de Trabalhos, são apresentados os indicadores dos trabalhos criados. Ao pressionar um desses indicadores, o usuário é redirecionado para uma página que contém informações detalhadas sobre o respectivo trabalho, bem como um quadro no formato Kanban, que permite a visualização e movimentação das tarefas vinculadas a ele, conforme ilustrado na Figura 17. Importante ressaltar que as tarefas possuem níveis de prioridade representados por cores no quadro Kanban.

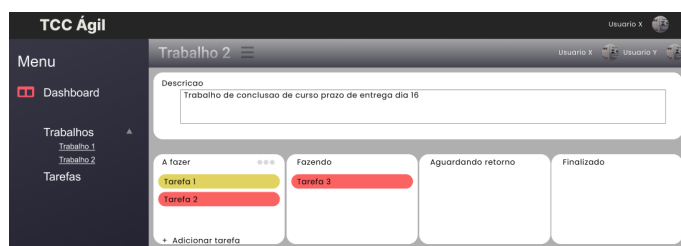


Fig. 17. Tela do Trabalho.

Para alterar o *status* das tarefas, o usuário deve selecionar a tarefa desejada e arrastá-la para o *status* correspondente. Para visualizar as informações detalhadas de uma tarefa, basta realizar um duplo clique, o que redirecionará o usuário para uma tela, representada pela Figura 18, que exibe informações como a descrição da tarefa, seu nível de prioridade (que só pode ser editado pelo orientador) e as atividades relacionadas. Essas atividades permitem ao usuário descrever ou inserir as informações sobre o que foi realizado, possibilitando um controle das alterações entre uma atividade e outra. Caso tenham sido realizadas alterações, o usuário deve pressionar o botão “Salvar”, localizado na parte inferior da tela, para salvar as modificações. Para retornar à tela do trabalho, basta pressionar o nome do trabalho, localizado na parte superior ou no menu à esquerda.

Fig. 18. Tela de Tarefa.

Na tela com as informações do trabalho, ilustrada na Figura 17, é possível criar uma nova tarefa pressionando o campo “Adicionar tarefa”, localizado na parte inferior do indicador de tarefas no *status* “A fazer”. O usuário será redirecionado para a tela representada na Figura 18, onde encontrará os campos a serem preenchidos. Após o preenchimento dos campos, basta clicar no botão “Salvar”.

Caso seja necessário excluir uma tarefa, o usuário encontrará a opção “Apagar Tarefa” no lado oposto do botão “Salvar”. Ao selecionar essa opção, será exibido um *pop-up* semelhante ao ilustrado na Figura 19 (que representa a exclusão de um trabalho), solicitando a confirmação antes de realizar a exclusão da tarefa.

Fig. 19. *Pop-up* para confirmar ação de exclusão de tarefa

Ainda na tela do trabalho, há um submenu representado por três linhas ao lado do nome do trabalho, localizado na parte superior da tela. Ao pressionar esse submenu, será exibida as opções “Adicionar Membro” e “Excluir Trabalho”. Ao selecionar “Excluir trabalho” será apresentado um *pop-up* na

tela do usuário para confirmar ou cancelar a exclusão do trabalho, semelhante ao da Figura 19.

Caso o usuário optar por pressionar o botão “Adicionar Membro” o usuário será redirecionado para a tela de convite de membros, conforme ilustrado na Figura 20. Basta informar o *e-mail* de um usuário existente para convidá-lo a participar do trabalho. Nessa tela, também é possível remover um membro do trabalho. Para salvar as alterações, é necessário pressionar o botão “Salvar”.

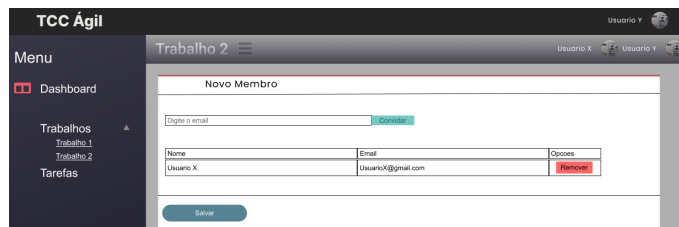


Fig. 20. Tela de Novo Membro

Para acessar a tela de perfil, representada na Figura 21, o usuário deve pressionar o nome de usuário ou o ícone, ambos localizados na parte superior direita das telas, conforme observado nas figuras anteriores. Ao fazer isso, o usuário será redirecionado para a tela de perfil. Nessa tela, é possível acompanhar um histórico de eventos, que vai desde a criação de um novo trabalho até a movimentação de uma tarefa para o *status* “Finalizada”. Além disso, é possível verificar a quantidade de trabalhos em que o usuário está participando e seu tipo de perfil (orientador ou orientando).



Fig. 21. Tela de Perfil

Por fim, há a tela de tarefas, representada na Figura 22, que pode ser acessada pressionando o botão “Tarefas” no menu à esquerda, presente em todas as telas. Essa tela tem o propósito de listar todas as tarefas, fornecendo informações como nome, descrição, prioridade, trabalho ao qual estão vinculadas e *status*. É possível ordenar as tarefas ao pressionar o nome das colunas. Para ordenar por nome, basta pressionar o campo “Nome”; para ordenar por *status*, é necessário pressionar o botão “Status”.

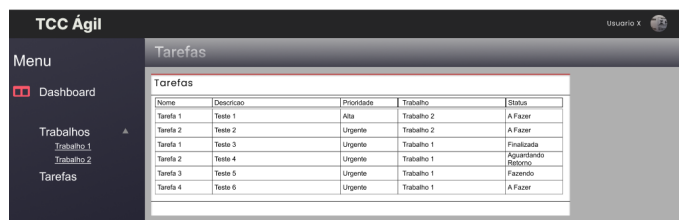


Fig. 22. Tela de Tarefas

Ainda na tela de tarefas, ao pressionar qualquer campo das informações das tarefas, o usuário será redirecionado para a tela correspondente. Por exemplo, ao pressionar a descrição, será redirecionado para a tela que contém as informações detalhadas da tarefa. Se pressionar o campo que indica o nome do trabalho ao qual a tarefa está vinculada, será direcionado para a tela que contém as informações do trabalho.

É importante ressaltar que as telas apresentadas são apenas protótipos e podem sofrer alterações com base no *feedback* dos usuários e iterações de *design*.

Essa aplicação *web* intuitiva e de fácil utilização beneficia a comunidade acadêmica ao facilitar o gerenciamento de trabalhos, otimizar o tempo e recursos disponíveis, promover uma comunicação eficiente entre orientador e orientado, e resultar em trabalhos de maior qualidade e produtividade.

V. DESENVOLVIMENTO

O desenvolvimento de aplicações *web* em Java é uma tarefa complexa e multifacetada, envolvendo a criação de uma infraestrutura robusta, integração eficiente com banco de dados, gerenciamento de autenticação, persistência de dados e implementação de controle de acesso. Nesta seção, serão explorados os principais aspectos desse processo, destacando como cada componente contribui para a construção de uma aplicação coesa e eficaz.

A. Banco de Dados

O desenvolvimento da aplicação *web* iniciou com a criação do banco de dados, seguindo o modelo do diagrama de entidade e relacionamento apresentado na Figura 7. Para garantir um controle mais preciso sobre as ações no banco, foram implementados *triggers* na tabela de Log. Um exemplo desses *triggers* pode ser visualizado na Figura 23, onde, após um *insert* na tabela *tcc.usuario*, são registradas as informações correspondentes na tabela de *log*.

A tabela de *Log* desempenha um papel crucial na aplicação, sendo utilizada, por exemplo, na construção da *timeline* na tela de perfil. Essa funcionalidade proporciona aos usuários uma visão dos eventos relacionados aos trabalhos nos quais estão envolvidos.

```
CREATE OR REPLACE FUNCTION cadastrar_usuario_log() RETURNS TRIGGER AS $$
BEGIN
    -- Insira um registro de log para o evento de cadastro de usuário
    INSERT INTO tcc.log (tipoEvento, descricaoEvento, dataHoraEvento, idUsuarioLog)
    VALUES ('Cadastro de Usuário', 'Novo usuário cadastrado: ' || NEW.nome, NOW(), NEW.idUsuario);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Crie a trigger para acionar a função quando um novo usuário é cadastrado
CREATE TRIGGER cadastrar_usuario_trigger
AFTER INSERT ON tcc.usuario
FOR EACH ROW
EXECUTE FUNCTION cadastrar_usuario_log();
```

Fig. 23. Trigger

B. Back-end

No que diz respeito ao *back-end* da aplicação, diversas tecnologias do ecossistema Spring, como o Spring Web, Spring Data JPA são empregadas para criar a infraestrutura de dados, repositórios e serviços, permitindo que o *back-end* processe os retornos dos *endpoints* e forneça páginas HTML para os navegadores dos usuários.

b.1) Integração com Banco de Dados

A integração eficiente com o banco de dados é simplificada pelo Spring JPA, que possibilita a configuração da conexão por meio do arquivo *application.properties*, conforme Figura 24. Nesse arquivo, são definidas informações sensíveis, como a URL, o nome de usuário e senha, sendo esta última protegida por questões de segurança.

```
tcc1 > src > main > resources > application.properties
You, 22 seconds ago | 1 author (You)
1 spring.datasource.url=jdbc:postgresql://localhost:5432/tccagil_apresentacao
2 spring.datasource.username=postgres
3 spring.datasource.password=*****
```

Fig. 24. Arquivo *application.properties*

b.2) Persistência de Dados com Classes de Entidades

A persistência de dados é conduzida por classes que representam cada tabela do banco de dados, como exemplificado na Figura 25, seguindo as melhores práticas de desenvolvimento. Essas classes, anotadas com JPA, possuem campos que correspondem diretamente às colunas das tabelas. A utilização de anotações como *@Entity* e *@Table* facilita a associação entre as classes e as entidades do banco de dados.

```
19 @Entity
20 @Table(name = "trabalho", schema = "tcc")
21 public class Trabalhos {
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     @Column(name = "idTrab")
25     private int idTrab;
26
27     public int getIdTrab() {
28         return idTrab;
29     }
30     @Column(name = "titulo")
31     private String titulo;
```

Fig. 25. Trecho de código da classe representante da tabela do banco de dados (Entidade)

Cada uma dessas classes inclui métodos *getters* e *setters*, conforme Figura 26, permitindo um acesso eficiente aos valores dos campos. Além disso, a presença de construtores em cada classe, permite a criação de instâncias a partir de objetos recebidos de outras partes do código facilitando a manipulação e inserção de dados no banco, tornando o código mais eficiente e coeso.

```
67 //Getter
68 public int getIdUsuario() {
69     return idUsuarioResponsavel;
70 }
71 //Setter
72 public void setIdUsuarioResponsavel(int idUsuarioResponsavel) {
73     this.idUsuarioResponsavel = idUsuarioResponsavel;
74 }
75 //Constructor
76 public TrabalhosDao(TrabalhosRecord dados) {
77     this.titulo = dados.tituloTrabalho();
78     this.descricao = dados.descricaoTrabalho();
79     this.dataDeInicio = dados.data_iniTrabalho();
80     this.prazoDeEntrega = dados.prazo_entregaTrabalho();
81     this.idUsuarioResponsavel = dados.idusuario_responsavel();
82 }
```

Fig. 26. Trecho de código de *getters*, *setters* e *constructors*

b.3) Gerenciamento de Autenticação

Para garantir a segurança da aplicação, o gerenciamento de autenticação é realizado pela classe *AutenticacaoService*, marcada, com a anotação *@Service*. Como um componente gerenciado pelo Spring, essa classe é injetada em diversas partes da aplicação, desempenhando funções essenciais, como verificar se um usuário está autenticado e agregar informações comuns a um modelo usado na aplicação *web*. Essa classe possui métodos, como:

- *isUsuarioLogado(HttpSession session)* que verifica se um usuário está autenticado, baseando-se na existência do atributo “*usuarioLogado*” na sessão *HttpSession*.
- *adicionarInformacoesComuns(Model model, HttpSession session)* que agrega informações comuns a um modelo usado na aplicação *web*.

```
17 @Service
18 public class AutenticacaoService {
19
20     @Autowired
21     private TrabalhosRepository trabalhosRepository;
22
23     public boolean isUsuarioLogado(HttpSession session) {
24         return session.getAttribute(name="usuarioLogado") != null;
25     }
26
27     public void adicionarInformacoesComuns(Model model, HttpSession session) {
28
29         UsuarioDao usuarioLogado = (UsuarioDao) session.getAttribute(name="usuarioLogado");
30         String nomeUsuario = usuarioLogado.getNome();
31         model.addAttribute(attributeName="nomeUsuario", nomeUsuario);
32
33         int idUsuario = (int) session.getAttribute(name="idusuarioLogado");
34
35         List<TrabalhosInfo> trabalhos = new ArrayList<>();
36         List<TrabalhosDao> trabalhosDoUsuario = trabalhosRepository.obterTrabalhosPorUsuario(idUsuario);
37         for (TrabalhosDao trabalho : trabalhosDoUsuario) {
38             TrabalhosInfo trabalhoInfo = new TrabalhosInfo(trabalho.getIdTrab(), trabalho.getIdTitulo());
39             trabalhos.add(trabalhoInfo);
40         }
41         model.addAttribute(attributeName="trabalhos", trabalhos);
42     }
43 }
```

Fig. 27. Trecho de código da Autenticação

b.4) Controllers

Os *controllers*, responsáveis por gerenciar as solicitações HTTP nas páginas HTML, utilizam a anotação *@Controller*, recebem as solicitações dos clientes, processam-nas e retornam as respostas apropriadas. A injeção de dependências, marcada com *@Autowired*, permite a integração de serviços ou repositórios nas propriedades das classes controladoras.

No contexto da Figura 28, o controlador injeta três dependências: *autenticacaoService*, *tarefasRepository* e *logRepository*. Essas dependências são, geralmente, serviços ou repositórios que a classe *PerfilController* utilizará para executar a lógica de negócios.

```
24 @Controller
25 public class perfilController {
26
27     @Autowired
28     private AutenticacaoService autenticacaoService;
29
30     @Autowired
31     private TarefasRepository tarefasRepository;
32
33     @Autowired
34     private LogRepository logRepository;
35
36     @GetMapping("/perfil")
37     public String perfil(HttpSession session, Model model) {
```

Fig. 28. Trecho de código para Injeção de *Repository*

b.5) Repository

No contexto dos *controllers*, os repositórios desempenham um papel fundamental. A anotação *@Repository* identifica as *interfaces* como repositórios *Spring*, responsáveis por operações de persistência de dados. A *interface* estende a *JpaRepository*, indicando que lida com operações de acesso a dados relacionadas à entidade correspondente.

No contexto da Figura 29 a anotação *@Repository* é empregada para identificar a *interface* *MembrosRepository* como um repositório *Spring*, responsável por operações de persistência de dados. A *interface* estende a *interface* *JpaRepository*, indicando que o *MembrosRepository* lida com operações de acesso a dados relacionadas à entidade *Membros*.

Além disso, a anotação *@Query* é utilizada para definir consultas personalizadas em JPQL (*Java Persistence Query Language*). O método *obterMembroPorTrabalho(@Param("idtrab") Long idtrab)* é um exemplo de operação personalizada que corresponde à consulta definida, permitindo a obtenção de membros com base no ID de um trabalho específico..

```
11 @Repository
12 public interface MembrosRepository extends JpaRepository<MembrosDao, Long> {
13
14     @Query("SELECT u.email as email, m FROM MembrosDao m JOIN UsuarioDao u ON m.usuarioId = u.idUsuario +
15           \"WHERE m.trabalhoId = :idtrab\")
16     List<Object[]> obterMembroPorTrabalho(@Param("idtrab") Long idtrab);
17
18     MembrosDao findByUsuarioIdAndTrabalhoId(int usuarioId, Long trabalhoId);
19 }
```

Fig. 29. Trecho de código da interface *Repository*

b.6) POST e GET

A comunicação eficiente entre clientes e servidores é facilitada pelos métodos HTTP POST e GET. O método GET é utilizado para solicitar dados de um recurso específico, sendo mais adequado para operações que não alteram o estado do servidor. Por outro lado, o método POST é empregado para enviar dados ao servidor, sendo a escolha adequada para operações que causam alterações no estado do servidor, como o envio de formulários. A combinação criteriosa desses métodos é vital para assegurar uma comunicação eficiente e segura, adaptando-se às necessidades específicas de cada operação.

Dando continuidade a essa eficiência na comunicação, a implementação de verificação de acesso estende-se além da autenticação de *login*, abrangendo as telas relacionadas à visualização e edição de trabalhos. A lógica de validação está presente nas classes de *controller*, garantindo que apenas usuários vinculados a determinado trabalho tenham acesso às páginas correspondentes.

Um exemplo dessa implementação é evidenciado na Figura 30, onde a condição está inserida no método GET da classe do *controller* que verifica a vinculação do usuário ao trabalho antes de liberar o acesso.

O método *trabalhoIndi* exemplifica essa abordagem, verificando se o usuário está vinculado ao trabalho antes de liberar o acesso. A lógica condicional dentro desse método valida a visibilidade do botão de remover trabalho, permitindo-o apenas para o criador do trabalho. Essa estratégia assegura que somente usuários autorizados, como criadores e membros

de determinado trabalho, tenham acesso às funcionalidades específicas relacionadas a esse trabalho.

A função primordial desse método é apresentar informações detalhadas sobre um trabalho específico e suas tarefas associadas. Para realizar essa tarefa, o código verifica o usuário logado, obtendo os dados necessários por meio da injeção de repositórios, garantindo uma abordagem coesa e eficiente na manipulação de informações.

```
36 @GetMapping("/trabalhoIndi/{idtrab}/tarefasCad")
37 public String novaTarefa(@PathVariable Long idtrab, HttpSession session, Model model) {
38     if (autenticacaoService.isUsuarioLogado(session)) {
39         autenticacaoService.adicionarInformacoesComuns(model, session);
40         int idUsuario = (int) session.getAttribute(name="idUsuarioLogado");
41
42         MembrosDao membrosInd = membrosService.obterMembroIDporTrabalho(idtrab, idUsuario);
43         TrabalhosDao trabalhoIndi = trabalhosIndService.obterTrabalhoPorId(idtrab);
44
45         model.addAttribute(attributeName="idUsuario", idUsuario);
46         model.addAttribute(attributeName="idtrab", idtrab); // You, last week - Feito em
47         // Verifica se o trabalho existe e se o usuário logado é o proprietário do
48         // trabalho
49         if (trabalhoIndi != null && (trabalhoIndi.getIdUsuario() == idUsuario
50         || (membrosInd != null && membrosInd.getIdUsuario() == idUsuario))) {
51             model.addAttribute(attributeName="trabalhoIndi", trabalhoIndi);
52             return "tarefasCad"; // Página para criar uma nova tarefa
53         } else {
54             return "redirect:/Acesso_negado";
55         }
56     } else {
57         return "redirect:/login";
58     }
59 }
```

Fig. 30. Condição de acesso aos trabalhos

A Figura 31 ilustra o trecho de código responsável pelo método POST utilizado no cadastro de usuários. Esse método é empregado em todos os *controllers* associados a páginas HTML que contenham formulários. Na tela de *login*, especificamente, é utilizado o método POST com um mapeamento específico para evitar conflitos entre os formulários de cadastro e *login* (*@PostMapping(params = "formAction=cadastro")*), uma vez que ambos estão presentes nessa página.

O método *public String cadastraUser(UsuarioRecord dados, HttpSession session)* é acionado em requisições POST, recebendo dois parâmetros: dados (um objeto da classe *UsuarioRecord* contendo informações do formulário) e *session* (representando a sessão do usuário).

O trecho *var userCadastro = new UsuarioDao(dados)* cria um novo objeto *UsuarioDao* com base nos dados fornecidos no formulário. Em seguida, *repository.save(userCadastro)* salva o novo usuário no repositório (banco de dados).

```
55 @PostMapping(params = "formAction=cadastro")
56 public String cadastraUser(UsuarioRecord dados, HttpSession session) {
57     // Verificar se o email já está cadastrado
58     if (repository.existsByEmail(dados.CadastroEmail())) {
59         return "redirect:/login?error=C.Email já cadastrado";
60     }
61     // Verificar se a senha e a confirmação de senha são iguais
62     if (!dados.CadastroPassword().equals(dados.CadastroConfirmPassword())) {
63         return "redirect:/login?error=C.Senhas não correspondem";
64     }
65     // Criar um novo objeto UsuarioDao com base nos dados recebidos
66     var userCadastro = new UsuarioDao(dados);
67     // Salvar o novo usuário no repositório (banco de dados)
68     repository.save(userCadastro);
69     // Definir um atributo de sessão para indicar que o cadastro foi bem-sucedido
70     session.setAttribute(name="cadastroSucesso", value="Usuário cadastrado com sucesso!");
71     return "login";
72 }
```

Fig. 31. Trecho de código do método *PostMapping*

Resumidamente, ao lidar com as requisições HTTP POST e GET, realiza-se verificações cruciais e são responsáveis por buscar, criar e armazenar informações no banco de dados. Cada classe incorpora sua lógica específica para processar

essas requisições, assegurando não apenas a eficácia na busca e armazenamento, mas também uma manipulação adequada dos dados preenchidos nos formulários, seja durante o envio de informações para o servidor (POST) ou a solicitação de dados específicos (GET).

Em síntese, cada componente desempenha um papel específico e essencial no desenvolvimento de aplicações *web* em Java. Desde a interação com o banco de dados ao controle de acesso, a abordagem integrada e coesa resulta em uma aplicação robusta e eficaz. Cada classe, método e componente contribui para a construção de uma experiência do usuário consistente e segura, refletindo a complexidade e a sofisticação do desenvolvimento de aplicações *web* em Java.

C. Front-end

O *front-end* da aplicação utiliza o Thymeleaf para construir as páginas HTML, integrando-se de maneira eficiente com o *back-end* em Java, uma vez que faz parte do ecossistema Spring.

Para garantir uma estilização padronizada nas páginas *web*, foi empregado o CSS (*Cascading Style Sheets*), proporcionando simplicidade e uniformidade no desenvolvimento das interfaces.

c.1) Tabelas

A exibição de tabelas de dados é aprimorada pelo *plugin DataTables* para jQuery. Esse *plugin* constrói dinamicamente as colunas e o conteúdo da página oferecendo recursos de ordenação, paginação e pesquisa de dados. Desempenhando um papel fundamental na página de Tarefas, conforme Figura 22, permitindo a ordenação das tarefas com base no cabeçalho da tabela.

Na Figura 32 é demonstrada a implementação da funcionalidade de ordenação de tabelas usando o *DataTables*. No elemento HTML `< head >`, são incluídos os arquivos CSS que definem os estilos para as tabelas formatadas pelo *DataTables*, assim como a biblioteca jQuery e o próprio *script* do *DataTables*, proporcionando recursos como classificação, pesquisa e paginação às tabelas HTML.

O bloco de *script* é responsável pela inicialização do *DataTables*, selecionando o elemento HTML com o “ID *tabela-tarefas*” e aplicando a funcionalidade *DataTables* a ele. Essa abordagem torna a tabela interativa, dotada de recursos como a classificação de colunas, pesquisa e paginação. A integração eficaz desses elementos contribui para uma experiência de usuário coesa e dinâmica na aplicação.

```

15 <link rel="stylesheet" type="text/css" href="https://cdn.datatables.net/1.13.7/css/jquery.dataTables.min.css">
16 <script type="text/javascript" charset="utf8" src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
17 <script type="text/javascript" charset="utf8" src="https://cdn.datatables.net/1.13.7/js/jquery.dataTables.min.js"></script>
18 </head>
19 <script>
20 $(document).ready(function() {
21     $('#tabela-tarefas').DataTable();
22 });
23 </script>

```

Fig. 32. Implementação *plugin DataTables*

c.2) Formulários

Na seção dedicada aos formulários é iniciada com a representação visual na Figura 33. Nele é observado o início do formulário, destacando o atributo “*method*” configurado como

“*post*”, indicando que os dados serão enviados ao servidor por meio do método HTTP POST. O atributo “*action*” especifica a URL para a qual os dados serão enviados, neste caso, “*/login?formAction=cadastra*”, sendo útil no lado do servidor para distinguir entre diferentes tipos de solicitações.

```

39 <div class="cadastro-form">
40 <h2>Cadastro</h2>
41 <form method="post" action="/login?formAction=cadastra">
42 <!-- metodo para enviar/cadastrar requisicao formAction para distinguir os Post
43     nas classe-->
44 <div class="form-group">
45 <label for="CadastroNome">Nome</label>
46 <input type="text" id="CadastroNome" name="CadastroNome" required>
47 </div>
48 <div class="form-group">
49 <label for="CadastroEmail">Email</label>
50 <input type="email" id="CadastroEmail" name="CadastroEmail" required>
51 </div>
52 <div class="form-group">
53 <label for="CadastroPassword">Senha</label>
54 <input type="password" id="CadastroPassword" name="CadastroPassword" required>
55 </div>
56 <div class="form-group">
57 <label for="CadastroConfirmPassword">Confirmar Senha</label>
58 <input type="password" id="CadastroConfirmPassword" name="CadastroConfirmPassword" required>
59 <div class="error-message">
60 <!-- Exiba mensagens de erro aqui, se necessário -->
61 <p th:text="${param.errorC}" style="color: red; font-size: 12px;"></p>
62 </div>
63 </div>
64 <div class="form-group">
65 <button type="submit" value="Cadastrar">Cadastrar</button>
66 </div>
67 </form>

```

Fig. 33. Trecho de código formulário HTML

Cada elemento do formulário segue princípios semelhantes de estrutura. Por exemplo, o rótulo associado ao campo de entrada de texto “*Nome*” é definido com `< label for = “CadastroNome” > Nome </label >`. O atributo “*for*” vincula o rótulo ao campo de entrada através do id correspondente. O campo de entrada para o nome, `< input type = “text” id = “CadastroNome” name = “CadastroNome” required >`, especifica o tipo de entrada (texto), apresenta um id único e um nome identificador para a transmissão dos dados. O atributo “*required*” indica que o preenchimento deste campo é obrigatório. Esses mesmos princípios se aplicam aos campos de *email*, senha e confirmação de senha.

A divisão `< div class = “error – message” > ... < /div >` é destinada à exibição de mensagens de erro, se necessário. Esta *div* pode conter uma mensagem de erro capturada pelo parâmetro “*errorC*”, sendo exibida em vermelho para alertar o usuário sobre possíveis problemas.

O botão de envio do formulário, `< button type = “submit” value = “Cadastrar” > Cadastrar < /button >`, desempenha o papel crucial de enviar os dados do formulário para o servidor quando acionado pelo usuário. Essa estrutura de formulário, com sua clareza e coesão, contribui para uma interação eficiente e intuitiva do usuário com a aplicação *web* Java proposta.

c.3) Apresentação de Dados nas Telas

A apresentação dos dados na interface ocorre por meio da iteração de funções no HTML para recuperar as variáveis definidas no *controller* da página correspondente, como ilustrado na Figura 34. Nessa implementação, destaca-se a criação de uma tabela HTML, conforme representado na mesma figura. A estrutura da tabela é definida com a seção `< thead >`, que inicia o cabeçalho da tabela.

Para preencher dinamicamente a tabela com a lista de membros (*\$membros*), utiliza-se a expressão Thymeleaf `th:each=“result : $membros”`. Dentro desse *loop*, a variável

local *result* é atribuída a cada membro da lista *\$membros*, permitindo o acesso às propriedades individuais de cada membro durante cada iteração. Isso facilita a construção dinâmica da tabela de membros.

Dentro do contexto da construção da tabela HTML, a expressão *th.text="result[1].nome"* utiliza a sintaxe Thymeleaf para incorporar dinamicamente o nome do membro em uma célula específica. A expressão *result[1].nome* recupera o valor do atributo “nome” do objeto associado ao índice 1 da lista de membros. Dessa forma, cada célula na coluna de nomes é preenchida dinamicamente durante a iteração pelos membros.

Os valores da lista de membros (*\$membros*) são provenientes do lado do servidor, mais especificamente do controlador (*controller*) da aplicação *web*. Quando há uma solicitação para exibir a página contendo a tabela de membros, o controlador acessa o modelo (*Model*) para obter os dados necessários. A lista de membros é recuperada do banco de dados ou de outra fonte de armazenamento de dados e adicionada ao modelo da página antes que a visualização seja renderizada.

```

48 <!-- Lista de membros já adicionados -->
49 <div class="form-group">
50 <h3>Membros Adicionados:</h3>
51 <table class="table_membro" style="margin-inline-start: 30px;">
52 <thead>
53 <tr>
54 <th class="th_membro">Nome</th>
55 <th class="th_membro">Email</th>
56 <th class="th_membro" style="width: 60px;">Opções</th>
57 </tr>
58 </thead>
59 <tbody>
60 <tr th:each="result : $membros">
61 <td class="td_membro" th:text="${result[1].nome}"></td>
62 <td class="td_membro" th:text="${result[0]}"></td>
63 <td class="td_membro" ><button type="button" class="remove-button"
64 th:attr="data-member-id=${result[1].idMembros}, data-work-id=${idtrab}">Remover</button>
65 </td>
66 </tr>
67 </tbody>
68 </table>
69 </div>

```

Fig. 34. Trecho de código utilizando objetos injetados

Essas expressões *Thymeleaf* desempenham um papel fundamental na dinamicidade e eficiência da apresentação dos dados na página, adaptando-se conforme os requisitos e usos nas telas. Esse método é amplamente utilizado em várias páginas HTML, proporcionando uma experiência de usuário personalizada e interativa.

c.4) Botão para Remover Trabalhos

A adição do botão de remoção de trabalhos é evidenciada na Figura 36, onde a implementação em *JavaScript* desempenha um papel fundamental. O trecho de código em questão tem como objetivo adicionar um ouvinte de evento a todos os botões de remoção presentes na página HTML, identificados pela classe “*remove-button*”.

Neste código, a *NodeList removeButtons* é utilizada para selecionar todos os elementos com a classe “*remove-button*”. Em seguida, um ouvinte de evento é adicionado a cada botão de remoção. Quando um botão é clicado, o *script* exibe um *pop-up* 35 de confirmação para o usuário. Se confirmado, o ID do trabalho é obtido do atributo “*data-work-id*” do botão pressionado. Posteriormente, uma solicitação HTTP POST é enviada para o servidor, indicando a remoção do trabalho específico. A resposta é tratada, exibindo um alerta de sucesso e

redirecionando o usuário para a página “/*trabalhos*” se o *status* for 200 (OK). Caso contrário, um alerta de erro é exibido.

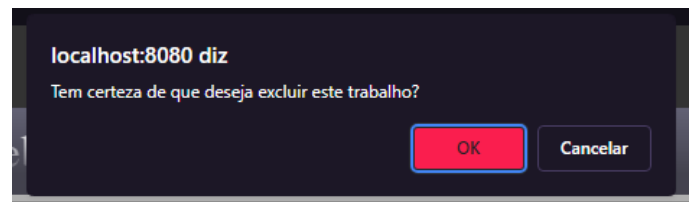


Fig. 35. Pop-up para confirmar ação de exclusão de trabalhos

Em resumo, esse *script* proporciona uma interação eficaz ao lidar com a remoção de trabalhos, solicitando confirmação ao usuário e, se confirmada, realizando a remoção no servidor.

```

109 <script>
110 // Adicione um ouvinte de evento a todos os botões de remoção
111 const removeButtons = document.querySelectorAll(".remove-button");
112 removeButtons.forEach(button => {
113   button.addEventListener("click", function () {
114     if (confirm("Tem certeza de que deseja excluir este trabalho?")) {
115       const workId = this.getAttribute("data-work-id");
116
117       // Envie uma solicitação para o servidor para remover o trabalho
118       fetch('/trabalhoIndi/${workId}/removeTrabalho', {
119         method: "POST",
120         headers: {
121           "Content-Type": "application/json"
122         },
123       })
124         .then(response => {
125           if (response.status === 200) {
126             alert("Trabalho Removido com sucesso.");
127             window.location.href = '/trabalhos';
128           } else {
129             alert("Erro ao remover o Trabalho.");
130           }
131         });
132     }
133   });
134 });
135 </script>

```

Fig. 36. Trecho de código para remover trabalhos

c.5) Arquivos HTML

O Thymeleaf possibilita a integração de objetos injetados pelos controladores com o HTML nativo. O Spring Boot configura automaticamente o diretório padrão como sendo “*src/main/resources*” para localizar as páginas HTML a serem retornadas pelos controladores nas requisições HTTP. Os arquivos devem ficar em lugares apropriados para serem encontrados, conforme Figura 37 em *resources* há uma pasta *static* contendo outras pastas como *css*, *template* e *img*, os códigos HTML devem ficar em *templates* para serem carregados de maneira correta.

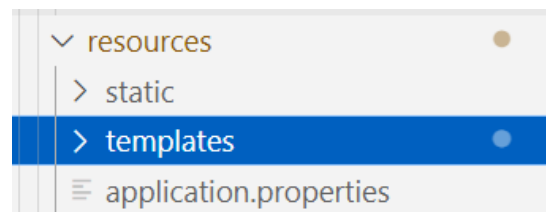


Fig. 37. Arquivos Thymeleaf

VI. RESULTADOS

Nesta seção será apresentado os resultados obtidos durante o processo de desenvolvimento da aplicação *web* proposta.

A. Sistema Web

A construção das páginas foi fortemente baseada nos protótipos do Figma, mantendo semelhanças significativas com as telas prototipadas.

O início da aplicação exibe a página de boas-vindas, apresentado na Figura 38, muito semelhante à tela de referência, com alterações mínimas no esquema de cores.



Fig. 38. Tela de Boas Vindas / Inicial

Em seguida, a tela de *login*, conforme Figura 39, responsável tanto pelo acesso quanto pelo cadastro, evita duplicatas de *e-mails* cadastrados. Ela segue o que foi prototipado, com a única mudança relacionada à exclusão do tipo de usuário do projeto por questões de usabilidade.

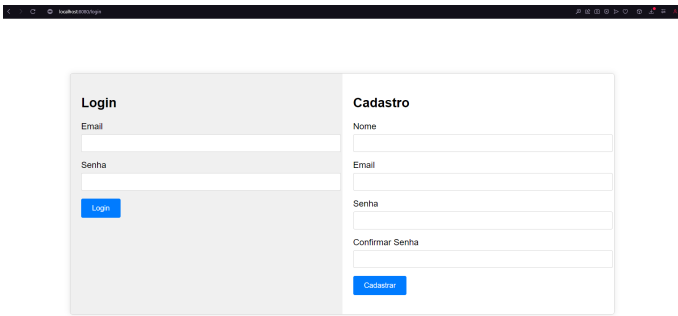


Fig. 39. Tela de login e de cadastro

Após o *login*, o usuário é redirecionado para a página de perfil, apresentada na Figura 40, que inclui a *timeline* e mostra informações atualizadas, como a quantidade de trabalhos e tarefas nas quais o usuário está envolvido.

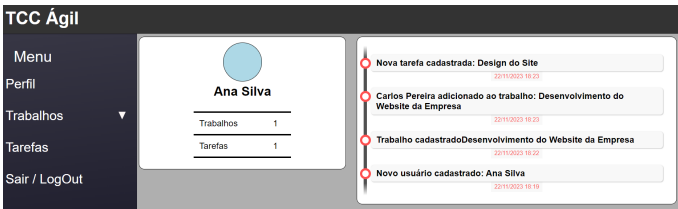


Fig. 40. Tela de perfil

A partir da tela de perfil, o usuário pode navegar entre as diferentes telas usando o menu à esquerda. Na página de trabalhos, ilustrada na Figura 41, o usuário visualiza os trabalhos nos quais participa e pode criar um novo trabalho.

Ao clicar no título do trabalho, é redirecionado para a página específica desse trabalho.

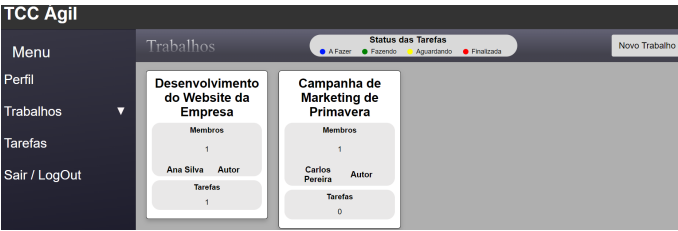


Fig. 41. Tela de *dashboard* dos trabalhos

Na tela do trabalho específico, conforme Figura 42, o usuário pode criar e editar tarefas, além de adicionar membros. A exclusão do trabalho é permitida apenas ao usuário que o criou, onde será apresentado um *pop-up* para confirmar a exclusão.

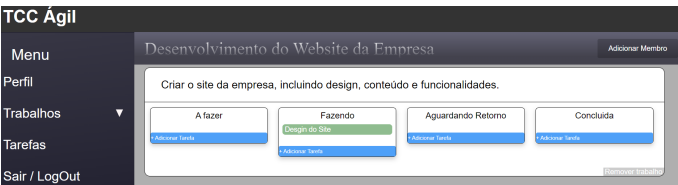


Fig. 42. Tela do Trabalho

As telas de edição e criação de tarefas, apresentada na Figura 43, são semelhantes, com a diferença de que a tela de edição possui campos adicionais para atividades. A edição da tarefa realiza uma atualização no banco de dados, permitindo a modificação do *status* e prioridade, enquanto as atividades só podem ser criadas.

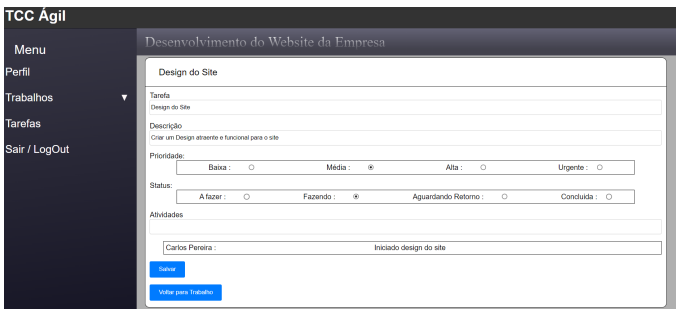


Fig. 43. Tela de edição de tarefas / criação de atividade

A tela de adição de membros, ilustrada na Figura 44, exibe os membros existentes e inclui um campo de autocompletar para adicionar novos membros.



Fig. 44. Tela de adição de membros

Por fim, a tela de tarefas, apresentada na Figura 45, contendo unicamente uma tabela que utiliza o *DataTables* para paginação, ordenação e pesquisa, permitindo que o usuário ajuste a exibição conforme sua necessidade.



Fig. 45. Tela Tarefas

VII. CONCLUSÃO

Este artigo apresentou a proposta e desenvolvimento de uma aplicação *web* voltada para o gerenciamento de Trabalhos de Conclusão de Curso (TCC) utilizando a metodologia ágil Kanban. Foi realizado um estudo detalhado sobre as dificuldades enfrentadas pelos alunos durante a elaboração dos TCCs e as ferramentas utilizadas para gerenciar esses trabalhos.

O levantamento de requisitos foi fundamental para compreender as necessidades dos usuários e orientar o desenvolvimento do sistema. Por meio de dois questionários aplicados ao público-alvo, foi possível obter uma aprovação de mais de 95% em relação à utilização de uma aplicação *web* para o gerenciamento dos TCCs. Ressalta-se que a visualização no formato de quadro Kanban foi a preferida pela maioria dos participantes.

Além disso, foram desenvolvidos diagramas e protótipos de tela, que permitiram uma visualização prévia da *interface* da aplicação e auxiliaram na validação do seu *design* e usabilidade. Essa etapa foi essencial para garantir a adequação da solução às expectativas dos usuários.

No processo de implementação da aplicação *web*, foram utilizadas tecnologias como Spring Boot, Spring Data e Thymeleaf, proporcionando agilidade nas configurações iniciais do projeto, otimização na interação com o banco de dados e facilitando a criação de páginas dinâmicas de maneira simplificada.

Os testes realizados evidenciaram a capacidade do sistema em atender aos principais requisitos especificados, validando a eficácia da aplicação proposta. Destaca-se a utilização do Spring Data para a implementação da *interface* JPA, ressaltando sua eficácia na criação rápida de consultas, bem como o ágil tratamento de consultas mais complexas com o auxílio do JPA, acelerando o processo de desenvolvimento.

A coesão do fluxo de navegação nas telas implementadas é atribuída à utilização de CSS, manipulação do HTML, e ao emprego pontual de JQuery e JavaScript. Destaca-se a responsividade da interface gráfica para telas de médio e grande porte.

O sistema, implementado com diversos dispositivos e linguagens como Java, PostgreSQL, Spring Boot, Thymeleaf, HTML e CSS, demonstrou, por meio dos testes realizados, a capacidade de atender aos principais requisitos definidos no decorrer do trabalho.

Ao longo do desenvolvimento, observou-se a necessidade de ajustes nas funções e requisitos inicialmente definidos. Essas alterações foram motivadas pela ênfase na acessibilidade e usabilidade da aplicação. Um exemplo notável é a decisão de eliminar a distinção entre usuários orientadores e orientandos. Essa modificação simplificou tanto a implementação quanto a criação de tabelas/entidades no banco de dados. Além disso, contribuiu significativamente para a usabilidade da aplicação, permitindo que um único usuário desempenhe tanto o papel de orientador quanto o de orientado. Essa abordagem aprimorou a praticidade da aplicação, cumprindo efetivamente com os objetivos propostos.

Diante dos resultados obtidos, a conclusão é que a proposta e o desenvolvimento da aplicação *web* são viáveis, atendendo às necessidades dos alunos e orientadores envolvidos na elaboração dos Trabalhos de Conclusão de Curso (TCCs). Tendo potencial para facilitar a interação entre os envolvidos, otimizar o fluxo de trabalho, melhorar a organização das tarefas e garantir o cumprimento dos prazos estabelecidos.

Para trabalhos futuros sugere-se os seguintes itens:

- Melhorar responsividade, recursividade e iteratividade das telas.
- Teste com os usuários finais para coletar *feedbacks* sobre o sistema.
- Autenticação: Poder fazer login usando *Google* ou outros meios.
- Implementar no sistema para ser colaborativo e simultâneo / colaboração em tempo real.
- Realização de um estudo sobre cores e contrastes para serem utilizados na aplicação, visando melhorar a visibilidade e agradar aos usuários.
- Criação de uma tela contendo dicas de formatação e referências conforme normas da ABNT.
- Manual de instruções de uso do sistema.
- Implementação de funcionalidade de envio de arquivos pelo sistema.
- Mover tarefas entre os quadros de status com o mouse.

REFERÊNCIAS

- [1] Janet Mesh. *Método Kanban: Guia detalhado e 5 modelos prontos para usar*. URL: <https://blog.trello.com/br/metodo-kanban> (acesso em 15/05/2023).
- [2] Thais Pacievitch. *Trabalho de Conclusão de Curso*. URL: <https://www.infoescola.com/educacao/trabalho-de-conclusao-de-curso> (acesso em 12/06/2023).
- [3] TCC Descomplicado. *O que é TCC? Entenda o Significado e os 7 Principais Tipos*. URL: <https://tccdescomplicado.com.br/o-que-e-tcc/> (acesso em 12/06/2023).
- [4] Wikipedia. *Trabalho de conclusão de curso*. 2022. URL: https://pt.wikipedia.org/wiki/Trabalho_de_conclus%C3%A3o_de_curso (acesso em 12/06/2023).
- [5] Daniel P. Araujo. *Metodologia Científica para Desenvolvimento de Software: Para cursos de Graduação e Pós-Graduação em TI*. Edição do Kindle., 2020. URL: <https://www.amazon.com.br/METODOLOGIA-CIENT%C3%8DFICA-para-Desenvolvimento-Software-ebook/dp/B087JPJHGY> (acesso em 12/05/2023).

- [6] PMI. *Um guia de conhecimento em gerenciamento de projetos*. 5ª ed. Saraiva, 2014. ISBN: 9788502223745.
- [7] Pinto Júnior Arnaldo Fernandes e Cristiana Fernandes De Muyllder. *O Sucesso na Implementação de Projetos Reflexões sobre lições aprendidas na literatura de Gerenciamento*. 2016. URL: https://pmiba.org.br/uploads/tinymce/files/pmi_bahia_submissao.pdf (acesso em 08/05/2023).
- [8] Alliance Agile. *Manifesto para Desenvolvimento Ágil de Software*. 2001. URL: <https://agilemanifesto.org/iso/ptbr/manifesto.html> (acesso em 12/05/2023).
- [9] José Henrique Teixeira de Carvalho Sbrocco e Paulo Cesar de Macedo. *Metodologias Ágeis - Engenharia de Software sob Medida*. Saraiva, 2012. ISBN: 9788536519418. URL: <https://integrada.minhabiblioteca.com.br/#/books/9788536519418/> (acesso em 12/05/2023).
- [10] FIA Business School. *Metodologias ágeis: o que são, tipos e principais vantagens*. 2018. URL: <https://fia.com.br/blog/metodologias-ageis/> (acesso em 14/05/2023).
- [11] MAX REHKOPF. *Kanban vs. Scrum: que tipo de ágil é você?* URL: https://www.atlassian.com/br/agile/kanban/kanban-vs-scrum?_ga=2.121302923.2043427570.1683557929-1961236859.1678740883 (acesso em 14/05/2023).
- [12] Agile Alliance. *Ágil 101*. 2023. URL: <https://www.agilealliance.org/agile101/> (acesso em 13/05/2023).
- [13] Henrik Kniberg e Mattias Skarin. *Kanban and Scrum making the most of both*. URL: http://www.agileinnovation.eu/wordpress/wp-content/uploads/2010/09/KanbanAndScrum_MakingTheMostOfBoth.pdf (acesso em 15/05/2023).
- [14] MAX REHKOPF. *O que é Kanban?* URL: <https://www.atlassian.com/agile/kanban> (acesso em 14/05/2023).
- [15] spring.io. *Why Spring?* URL: <https://spring.io/why-spring> (acesso em 12/03/2023).
- [16] baeldung. *Learn Spring and Spring Boot*. URL: <https://www.baeldung.com/learn-spring-course> (acesso em 12/03/2023).
- [17] Cleyson Lima. *Método Kanban: Guia detalhado e 5 modelos prontos para usar*. URL: <https://www.treinaweb.com.br/blog/o-que-e-o-spring-boot> (acesso em 15/05/2023).
- [18] Ramesh Fadatare. *Getting Started with Spring Boot*. URL: <https://www.javaguides.net/2018/09/getting-started-with-spring-boot.html> (acesso em 15/05/2023).
- [19] Willian. *Primeiros passos com o Spring Boot*. URL: <https://www.devmedia.com.br/primeiros-passos-com-o-spring-boot/33654> (acesso em 15/05/2023).
- [20] thymeleaf.org. *Tutorial: Using Thymeleaf*. URL: <https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html> (acesso em 11/09/2023).
- [21] mozilla.org. *JavaScript*. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (acesso em 11/09/2023).
- [22] The PostgreSQL Global Development Group. *About*. URL: <https://www.postgresql.org/about/> (acesso em 15/05/2023).
- [23] DevMedia. *Tecnologia PostgreSQL*. URL: <https://www.devmedia.com.br/guia/tecnologia-postgresql/34328> (acesso em 15/05/2023).
- [24] Michele Lopes. *O que é Figma e como usar?*. URL: <https://ebaonline.com.br/blog/o-que-e-figma-e-como-usar> (acesso em 09/06/2023).
- [25] Figma. *Design*. URL: <https://www.figma.com/design/> (acesso em 09/06/2023).
- [26] Roger S. PRESSMAN. *Engenharia de software : uma abordagem profissional*. AMGH, 2021. ISBN: 9786558040118. URL: <https://integrada.minhabiblioteca.com.br/reader/books/9786558040118> (acesso em 04/06/2023).
- [27] lucidchart. *O que é diagrama de atividades UML?*. URL: <https://www.lucidchart.com/pages/pt/o-que-e-diagrama-de-atividades-uml> (acesso em 07/06/2023).
- [28] IBM. *Diagramas de Atividades*. URL: <https://www.ibm.com/docs/pt-br/rational-soft-arch/9.7.0?topic=diagrams-activity> (acesso em 07/06/2023).

Modelo do Termo de autorização.

Termo de Ciência e Autorização para Publicação Eletrônica na Biblioteca Digital da Universidade do Oeste de Santa Catarina – Unoesc

1. Identificação do material bibliográfico:

- (X) TCC Graduação; (Monografia, artigo e outros)
- () TCC Pós-graduação Especialização;
- () Relatório de Estágio
- () Dissertação;
- () Tese.

2. Identificação do autor : Paulo Ricardo Savaris dos Santos

Curso: Engenharia de Computação

Título do Trabalho: Desenvolvimento de uma aplicação web para gerenciamento de trabalhos acadêmicos

E-mail: pauloosavaris@gmail.com

Orientador: Kleyton Hoffmann

e-mail: kleyton.hoffmann@unoesc.edu.br

Número de páginas: 18

Data de defesa: 28/11/2023 Data de entrega do arquivo: 08/12/2023

3. Informações de acesso ao documento:

Este trabalho é confidencial?¹ () Sim (X) Não

Pode ser liberado para publicação na Biblioteca Digital (X) Total () Parcial

Em caso de publicação parcial, assinale as permissões:

- () Sumário
 - () Resumo
 - () Bibliografia
 - () Outras permissões, quais?
-

Na qualidade de titular dos direitos de autor da publicação supracitada, de acordo com a Lei nº 9610/98, autorizo a Universidade do Oeste de Santa Catarina – Unoesc, a disponibilizar gratuitamente, sem ressarcimento dos direitos autorais, conforme permissões assinadas acima, do documento, em meio eletrônico, na Rede Mundial de Computadores, no formato especificado², para fins de leitura, impressão e/ou download pela Internet, a título de divulgação da produção científica gerada pela Universidade, a partir desta data.

4. Está sujeito a registro de patente?

(X) Não

() Sim. Informar o nº do processo de encaminhamento ao Escritório de Interação e Transferência de Tecnologia. _____

Declaro ser de minha responsabilidade a autoria do texto referente ao Trabalho de Conclusão de Curso.

Joaçaba, 08/12/2023

Paulo Ricardo S. S
Assinatura do autor (es)

¹ Esta classificação poderá ser mantida por até um ano a partir da data de defesa. A extensão deste prazo suscita justificativa junto à Coordenação do Curso. Todo resumo estará disponível para reprodução, conforme Regulamento do Programa de Pós-graduação da Unoesc.

² Texto (PDF); imagem (JPG OU GIF); som (WAV, MPEG, AIFF, SND); vídeo (MPEG, AVI, QT), outros (específico da área).

Os dados pessoais solicitados neste formulário serão utilizados apenas no processo de publicação eletrônica da biblioteca digital da Unoesc.

A qualquer momento, o titular dos dados poderá consultar os dados pessoais tratados, bem como os prazos legais para seu armazenamento, mediante solicitação por e-mail ou por escrito remetida ao Encarregado de Proteção de dados, conforme artigo 8º, §5, da Lei nº 13.709/18 (LGPD).

Este termo de autorização deve ser digitalizado e anexado na parte final do trabalho entregue ao curso.